

Clawer C Data Normalization Engine

C 語言大學排名資料正規化引擎

將異質爬蟲輸出 (名稱 / 國家 / 排名 / 分數) 轉換為可直接比較的標準化紀錄

TEAM · 組員

b11402037

高恩在

b11402053

林秉學



```
$ make run
==== Clawer C Data Normalization Engine ====
1. 載入 CSV    3. 正規化    5. 匯出

$ make test_all
Passed: 602 / 602 – All tests passed.
```

簡介 — 背景、動機與研究問題

■ 背景與動機

上游爬蟲系統自 QS、THE、Shanghai Ranking 等多個排名來源抓取資料，輸出格式高度異質，無法直接進行跨來源比較與分析。

大學名稱 MIT · "University of California... (UCB)"

大小寫不一、縮寫、標點與空白差異

國家名稱 U.S.A. · UK · PR China · The Netherlands

縮寫、官方長名、「The X」前綴變體

排名格式 53 · 101-150 · Top 100 · =201 · #10 · 201+

純數字、區間、多種文字前綴

分數欄位 " 91.7 " · ".5" · "" · "N/A"

空白、前導小數點、無效值

■ 研究問題

在不修改下游分析邏輯的前提下，如何將異質性爬蟲輸出轉換為可直接比較的數值紀錄？

■ 設計選擇 — 為何用 C？

- 以 **C11** (`gcc -Wall -Wextra`) 實作核心引擎，而非 Python / pandas
- 大量 CSV 批次處理時維持低記憶體佔用與可預期的執行時間
- 無 Python 執行期依賴，便於與上游爬蟲管線部署整合
- 單機、檔案導向：無資料庫伺服器、無 Web API、無網路依賴

資料庫設計 — CSV Schema 與資料流

採檔案導向設計：以 CSV 作為持久層，不依賴外部資料庫伺服器，降低部署複雜度並貼合爬蟲批次輸出情境。

輸入 — 爬蟲原始 CSV (11 欄)

```
QS Rank,University,Country,GMAT,GRE,  
GPA,IELTS,TOEFL,Duolingo,  
Overall Score,URL
```



正規化

輸出 — 標準化 CSV (5 欄)

```
University,Country,  
Rank Min,Rank Max,  
Overall Score
```

實際資料範例 (前 → 後)

```
raw | University of California Berkeley (UCB) , usa ,12 , 14 , 91.7  
norm | University of California Berkeley (Ucb),United States,12,14,91.70
```

- Reader 防衛機制：header 欄位數與欄名驗證（不符即拒絕載入）、RFC 4180 quoted CSV 解析、UTF-8 BOM 清理、不合法列略過
- Writer 輸出保障：必要時自動 quoting / escaping，數值欄固定 %.2f 格式

資料庫設計 — 核心資料結構與對照表

```
/* include/record.h */
typedef struct {
    /* Raw input fields */
    char raw_name[150];
    char raw_country[80];
    char raw_rank[50];
    char raw_score[50];
    /* Normalized fields */
    char normalized_name[150];
    char normalized_country[80];
    int    rank_min, rank_max;
    double score;
} UniversityRecord;
```

raw 與 normalized 欄位並存於同一筆紀錄，便於前後對照、預覽與除錯

國家別名對照表 mapping_table[]

靜態 key → value 查表：先 normalize_basic (去標點、轉小寫、壓縮空白) 再比對；涵蓋縮寫 (U.S.A.)、官方長名 (Federal Republic of Germany) 與「The X」變體，38+ 條目 (Session 6 擴充)。

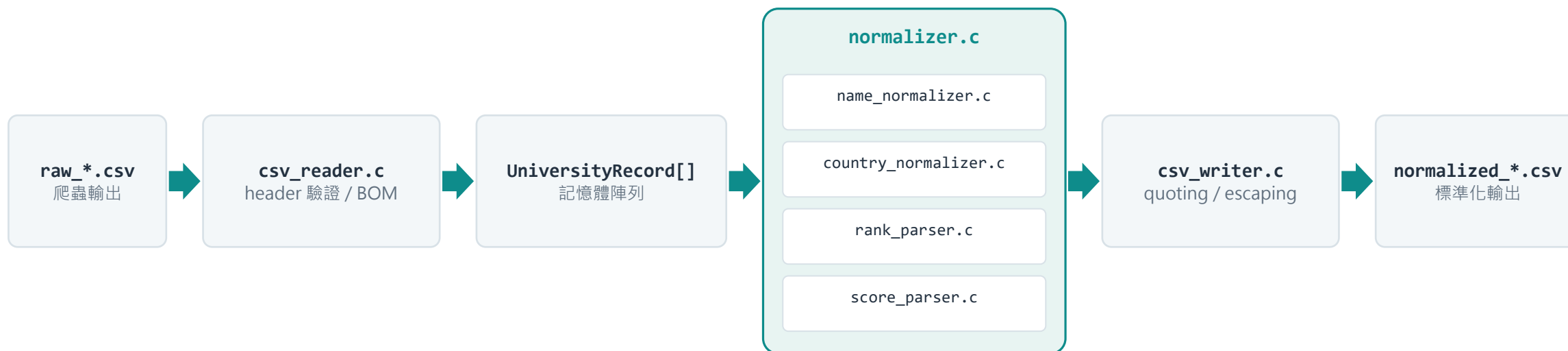
排名區間表示 rank_min / rank_max

以兩個整數欄位統一表達單值與區間：53 → [53,53]；101-150 → [101,150]；Top 100 → [1,100]，下游可直接做數值比較。

哨兵值設計 score = -1

無法解析的分數標記為 -1，由 writer 層轉為空欄位輸出，避免髒資料混入下游統計。

實作 — 系統架構與模組分工



互動式 CLI — `src/main.c`

- 1 載入 CSV
- 2 顯示原始
- 3 執行正規化
- 4 預覽結果
- 5 匯出 CSV
- 0 離開

狀態防呆：未載入 / 未正規化時擋下後續操作

共用工具與建置

`utils.c`：trim、collapse spaces、title case、縮寫 (MIT / ETH / KAUST) 偵測

`Makefile`：make run 互動執行 · make test 單元測試 · 測試套件一鍵建置

實作 — 正規化規則與實例

模組	原始輸入 (raw)	正規化輸出 (normalized)
名稱	University of California Berkeley (UCB)	University of California Berkeley (Ucb)
名稱	KAUST · hkust · eth	KAUST · HKUST · ETH (知名縮寫保留全大寫)
國家	usa · U.S.A. · UK	United States · United Kingdom
國家	The Netherlands · PR China	Netherlands · China (Mainland)
排名	Top 100 · #10 · =201	[1,100] · [10,10] · [201,201]
排名	101-150 · 150-101	[101,150] (反轉區間自動交換)
分數	" 91.7 " · ".5"	91.70 · 0.50 (前導小數點修正)
分數	"" · "N/A" · "abc"	-1 (哨兵值 → 輸出層轉空欄)

規則涵蓋 6 種排名格式、大小寫不敏感比對 ("TOP 100" ≡ "Top 100") ，並拒絕負數排名、TOP 0 與整數溢位 (strtol + 上限防護) 。

實作 — 弱點導向測試與驗證

■ 測試方法論 (每輪 Session)

- 1

靜態分析

通讀模組程式碼，標記可疑邏輯
- 2

探針程式

撰寫 probe 重現可疑行為
- 3

修正

最小修改修復根因
- 4

回歸驗證

新增 regression 套件全量重跑

累計 6 輪 Session、17 項修正 (A-T)
例：Fix S - parse_score(".5") 誤判為 5.0 → 修正為 0.5

602 / 602

九個測試套件全數通過 (make test_all)

test_normalizer	17 / 17
test_extreme	139 / 139
test_scale	31 / 31
test_weakness	43 / 43
test_regression2 - 6	372 / 372
合計	602 / 602

結論 — 成果、限制與未來工作

主要成果

- 完成 C11 正規化引擎：reader → 四大解析模組 → writer 完整 pipeline
- 602 項單元 / 回歸測試全數通過，涵蓋極端值、規模與弱點情境
- 6 輪弱點導向修正 (A–T 共 17 項) 建立回歸防護網

已知限制

- ASCII-only：É / ä / ã 等非 ASCII 字元直接傳遞，不做 Unicode 正規化
- requirement_parser 尚未實作 (GMAT / GRE / GPA / IELTS / TOEFL / Duolingo)
- %.2f 捨入：99.999 輸出為 100.00 (超出滿分)

未來工作

- 引入 utf8proc / ICU 處理非 ASCII 字元正規化
- 國家別名表外部化：從 mapping 檔載入，取代硬編碼
- CI 整合 (爬蟲 → 引擎 → 資料庫) 並提供 libnormalizer 函式庫介面

以小而可驗證的 C 核心，換取資料管線中可預期的正規化行為。

參考文獻

■ 排名資料來源

- QS World University Rankings — topuniversities.com
- Times Higher Education (THE) World University Rankings
- ShanghaiRanking / Academic Ranking of World Universities (ARWU)

■ 語言與工具

- ISO/IEC 9899:2011 — C11 Language Standard
- GCC Compiler Documentation (`-Wall -Wextra`)
- GNU Make Manual

■ 技術標準

- ISO 3166-1 — 國家代碼標準 (國家別名映射依據)
- RFC 4180 — CSV 格式標準 (quoting / escaping 規範)
- Unicode UAX #15 — Unicode Normalization Forms (NFC / NFD)

■ 專案內部文件

- README.md — 使用說明與驗證流程
- `c_engine/docs/architecture.md` — 系統架構與資料流
- `test-report.md` — Session 1-6 累計測試報告